CHAPTER 5

Borrows in Practice: Reborrows, Temporaries and the Limits of NLL

The first four chapters built a model: a lifetime is a region of the program (Chapter 1), a type that holds a borrow carries that region as a parameter (Chapter 2), elision writes the regions of a signature for you (Chapter 3), and the outlives order makes regions a subtyping relation (Chapter 4). That model explains why a program is accepted or rejected. It does not yet explain some things you see every day.

You pass the same `&mut Vec<String>` to a function twice in a row, although a `&mut` reference cannot be copied. You write `v.push(v.len())`, which borrows `v` mutably and immutably at once, and the compiler says nothing. You add one method call to a line, `String::from("x").as_str()`, and get an error about a "temporary value". You give a struct a `Drop` impl, and code that compiled a minute ago is rejected with a note about drop order. And now and then the compiler rejects a function that is plainly correct.

This chapter is about the borrow checker as you meet it in practice: the mechanics it applies without telling you (reborrows, two-phase borrows), the values whose lifetime you do not see in the source (temporaries, drops), and the known cases where it is more conservative than it needs to be.

In this chapter you will learn:

- what a reborrow is, when the compiler inserts one, and why `let r2 = r;` moves a `&mut` reference while `f(r)` does not;
- how two-phase borrows make `v.push(v.len())` compile, and where they stop;
- how to hold two mutable borrows into one value at once (fields, `split_at_mut`, `get_disjoint_mut`), and why accessor methods cannot do it;
- what E0505, E0515 and E0716 mean, and their standard fixes;
- when temporaries are dropped, what lifetime extension does, and what edition 2024 changed for tail expressions and `if let`;
- how drop order and the drop check decide whether a value with a `Drop` impl may hold a borrow;
- which correct programs the current borrow checker still rejects, and what Polonius changes.

5.1 Reborrowing

A `&mut` reference is not `Copy`

A shared reference `&T` is `Copy`: duplicating it creates a second reader, which is harmless. A mutable reference `&mut T` is not, because its whole point is to be the *only* path to the data while it is live. Two copies would be two writers. So assigning a `&mut` to another variable moves it, as with any non-`Copy` value. The full program is in `code/compile_fail/ch05_move_mut_ref.rs`, and the compiler rejects it:

```
pub fn collect(words: &mut Vec<String>) {
    let other = words; // moves the reference out of `words`
    add_word(other, "alpha");
    add_word(words, "beta");
}
```

```
error[E0382]: borrow of moved value: `words`
  --> compile_fail/ch05_move_mut_ref.rs:14:14
   |
11 | pub fn collect(words: &mut Vec<String>) {
   |                ----- move occurs because `words` has type `&mut Vec<String>`, which
does not implement the `Copy` trait
12 |     let other = words; // moves the reference out of `words`
   |                 ----- value moved here
13 |     add_word(other, "alpha");
14 |     add_word(words, "beta");
   |              ^^^^^ value borrowed here after move
```

The three labels tell the whole story. The first explains why there is a move at all: `words` has type `&mut Vec<String>`, "which does not implement the `Copy` trait". The second marks the move (`let other = words`), and the third the use after it. Nothing here is about lifetimes; it is ordinary ownership, applied to a reference.

Implicit reborrows

Now look at a function that uses its `&mut` parameter twice. It compiles (the program is `code/examples/ch05_reborrow.rs`):

```
fn add_word(words: &mut Vec<String>, word: &str) {
    words.push(word.to_owned());
}

fn collect(words: &mut Vec<String>) {
    add_word(words, "alpha"); // really `add_word(&mut *words, "alpha")`
    add_word(words, "beta"); // `words` is usable again: only a reborrow was passed
}
```

If `add_word(words, "alpha")` moved `words` into the call, the second call would be the same error as above. It is not, because the compiler does not pass `words` itself: it passes a **reborrow**, a new mutable reference to the same place, written `&mut *words`. You dereference the reference (`*words` is the `Vec<String>` it points to) and borrow that place again. The new reference has its own region, which may be shorter than the region of `words` and lasts here only for the call. While the reborrow is live, `words` is frozen; when it ends, `words` is usable again.

The compiler inserts a reborrow wherever it knows that the expected type is a `&mut` reference: an argument for a parameter declared `&mut T`, a `let` with a written `&mut` type, the receiver of a method

that takes `&mut self`. The same `let` that moved `words` above reborrows it as soon as you write the type:

```
fn collect_more(words: &mut Vec<String>) {
    let other: &mut Vec<String> = words; // the type is written: a reborrow
    add_word(other, "gamma");
    add_word(words, "delta"); // fine: `other` is no longer used

    let explicit = &mut *words; // the same reborrow, spelled out
    explicit.push(String::from("epsilon"));
    words.sort(); // a method call reborrows its receiver too
}
```

The three forms (`let other: &mut Vec<String> = words`, `&mut *words` written by hand, and the implicit `&mut *words` of `words.sort()`) are the same operation.

Where no reborrow happens

The rule has a consequence that surprises everyone once: when the expected type is a type parameter, the compiler does not know that it is a reference, so it does not reborrow. It moves. From `code/compile_fail/ch05_generic_moves.rs`:

```
fn passthrough<T>(value: T) -> T {
    value
}

pub fn generic_call(words: &mut Vec<String>) {
    passthrough(words).push(String::from("zeta"));
    passthrough(words).push(String::from("eta"));
}
```

```
error[E0382]: use of moved value: `words`
  --> compile_fail/ch05_generic_moves.rs:12:17
   |
10 | pub fn generic_call(words: &mut Vec<String>) {
   |                     ----- move occurs because `words` has type `&mut Vec<String>`,
which does not implement the `Copy` trait
11 |     passthrough(words).push(String::from("zeta"));
   |                 ----- value moved here
12 |     passthrough(words).push(String::from("eta"));
   |                 ^^^^^ value used here after move
   |
help: consider creating a fresh reborrow of `words` here
   |
11 |     passthrough(&mut *words).push(String::from("zeta"));
   |                 ++++++
```

`T` is inferred as `&mut Vec<String>`, and the reference is moved into the first call. This time the compiler suggests the fix itself: "consider creating a fresh reborrow of `words` here", that is, `passthrough(&mut *words)`, which is what `ch05_reborrow.rs` does:

```
fn passthrough<T>(value: T) -> T {
    value
}

fn generic_call(words: &mut Vec<String>) {
    passthrough(&mut *words).push(String::from("zeta")); // explicit reborrow
    passthrough(&mut *words).push(String::from("eta"));
}
```

The same happens when the `&mut` is wrapped in another type. An `Option<&mut u32>` is not a reference, so passing it moves it (`code/compile_fail/ch05_option_mut_moves.rs`):

```
fn bump(counter: Option<&mut u32>) {
    if let Some(count) = counter {
        *count += 1;
    }
}

pub fn bump_twice(counter: Option<&mut u32>) {
    bump(counter);
    bump(counter);
}
```

```
error[E0382]: use of moved value: `counter`
  --> compile_fail/ch05_option_mut_moves.rs:14:10
   |
12 | pub fn bump_twice(counter: Option<&mut u32>) {
   |                   ------- move occurs because `counter` has type `Option<&mut u32>`,
which does not implement the `Copy` trait
13 |     bump(counter);
   |          ------- value moved here
14 |     bump(counter);
   |          ^^^^^^^ value used here after move
   |
note: consider changing this parameter type in function `bump` to borrow instead if owning
the value isn't necessary
  --> compile_fail/ch05_option_mut_moves.rs:6:18
   |
 6 | fn bump(counter: Option<&mut u32>) {
   |    ----          ^^^^^^^^^^^^^^^^ this parameter takes ownership of the value
   |    |
   |    in this function
```

Here the note points in an unhelpful direction: changing `bump` to take a reference to the `Option` would work, but it changes the function's contract for every caller. The local fix is to reborrow what is inside the `Option` : `counter.as_deref_mut()` borrows `counter` for a moment and returns a new `Option<&mut u32>` that reborrows the same `u32` :

```
fn bump(counter: Option<&mut u32>) {
    if let Some(count) = counter {
        *count += 1;
    }
}

fn bump_twice(mut counter: Option<&mut u32>) {
    bump(counter.as_deref_mut()); // reborrow what is inside the `Option`
    bump(counter.as_deref_mut());
}
```

A reborrow freezes the original

While a reborrow is live, the reference it came from cannot be used, not even to read: that would be a second path to the same data (`code/compile_fail/ch05_reborrow_frozen.rs`):

```
pub fn collect(words: &mut Vec<String>) {
    let other = &mut *words;
    words.push(String::from("alpha"));
    other.push(String::from("beta"));
}
```

```
error[E0499]: cannot borrow `*words` as mutable more than once at a time
 --> compile_fail/ch05_reborrow_frozen.rs:8:5
  |
7 |     let other = &mut *words;
  |                 ----------- first mutable borrow occurs here
8 |     words.push(String::from("alpha"));
  |     ^^^^^ second mutable borrow occurs here
9 |     other.push(String::from("beta"));
  |     ----- first borrow later used here
```

Read the first line carefully: the conflict is on `*words`, the vector, not on the variable `words`. Borrows are tracked per *place*. `other` borrows `*words` from line 7 until its last use on line 9, and `words.push` (which reborrows `*words` again for the call) falls inside that region. Swap lines 8 and 9 and the program compiles, because `other` is dead by the time `words` is used.

This is also how a function that returns a `&mut` works. The result of

```
/// The result is a reborrow of `*words`: it may live as long as the caller's
/// borrow, but no longer.
fn longest_mut(words: &mut [String]) -> Option<&mut String> {
    words.iter_mut().max_by_key(|w| w.len())
}
```

is a reborrow of `*words` whose region the caller decides, within the region of the caller's own borrow. While the caller holds the returned `&mut String`, its vector is frozen. You met this in Chapter 2 as "the result pins the parser": it is the same mechanism.

The output of the example confirms that every call went through:

```
["alpha", "beta", "delta", "epsilon", "gamma", "zeta", "eta"]
["alpha", "beta", "delta", "EPSILON", "gamma", "zeta", "eta"]
count = 2
```

5.2 Two-phase borrows

Method calls evaluate the receiver before the arguments. So `lengths.push(lengths.len())` means: borrow `lengths` mutably for `push`, then evaluate `lengths.len()`, which borrows it immutably, then call `push`. By the rules of Chapter 1 that is a shared borrow inside a mutable one, error E0502. Yet this compiles (`code/examples/ch05_two_phase.rs`):

```
let mut lengths = vec![3, 1];
lengths.push(lengths.len()); // `&mut lengths` reserved, `lengths.len()` reads

let mut ids: HashMap<String, usize> = HashMap::new();
ids.insert(String::from("alpha"), ids.len());
ids.insert(String::from("beta"), ids.len());

let mut text = String::from("ab");
text.push_str(&text.to_uppercase()); // the shared borrow ends before the call

let first = &lengths[0];
lengths.push(*first); // `*first` is copied out before the push is activated
```

```
lengths = [3, 1, 2, 3, 4]
ids = [("alpha", 0), ("beta", 1)]
text = abAB
total = Total(10)
```

The rule that accepts it is called **two-phase borrows**. Some mutable borrows are created in two steps. When the receiver is evaluated, the `&mut` is only *reserved*; it is *activated* when the call actually starts. Between the two, the borrow behaves like a shared borrow, so other code may read the value but not modify it. In `lengths.push(lengths.len())`, the read happens during the reservation, finishes, and only then is the borrow activated.

The last two lines of the listing show what "finishes" means. In `text.push_str(&text.to_uppercase())`, the shared borrow `&text` is used by `to_uppercase`, which returns a new `String`; the argument passed to `push_str` borrows that new string, not `text`. In `lengths.push(*first)`, the expression `*first` copies the number out of the vector during the reservation, and `first` is never used again. In both cases no shared borrow of the receiver is live when the mutable one is activated.

The rustc development guide lists the three places where two-phase borrows are created: https://rustc-dev-guide.rust-lang.org/borrow-check/two-phase-borrows.html. The method-call receiver is the first. The second is the implicit reborrow of a `&mut` argument, from the previous section:

```
fn record(lengths: &mut Vec<usize>, value: usize) {
    lengths.push(value);
}

fn record_count(lengths: &mut Vec<usize>) {
    record(lengths, lengths.len()); // the implicit reborrow is two-phase too
}
```

The third is the `&mut` borrow of the left operand of an overloaded compound assignment operator, such as `+=` on a type that implements `AddAssign`:

```
#[derive(Debug)]
struct Total(usize);

impl std::ops::AddAssign<usize> for Total {
    fn add_assign(&mut self, amount: usize) {
        self.0 += amount;
    }
}
```

```
let mut total = Total(5);
total += total.0; // `AddAssign::add_assign(&mut total, total.0)`, two-phase
```

Where two-phase borrows stop

A `&mut` that you write yourself is an ordinary borrow, active immediately. Spell the same call in function syntax and it is rejected (`code/compile_fail/ch05_two_phase_explicit.rs`):

```
pub fn record_length(mut lengths: Vec<usize>) -> Vec<usize> {
    Vec::push(&mut lengths, lengths.len());
    lengths
}
```

```
error[E0502]: cannot borrow `lengths` as immutable because it is also borrowed as mutable
 --> compile_fail/ch05_two_phase_explicit.rs:7:29
  |
7 |     Vec::push(&mut lengths, lengths.len());
  |     --------- ------------  ^^^^^^^ immutable borrow occurs here
  |     |         |
```

```
  |     |         mutable borrow occurs here
  |     mutable borrow later used by call
```

The labels lay out the order of evaluation: the mutable borrow `&mut lengths` first, then the immutable borrow in `lengths.len()` while it is active, and "mutable borrow later used by call" to show why the first borrow is still live.

The reservation also does not excuse a shared borrow that is still *in use* when the call starts. `text.push_str(&text)` would append a string to itself; the argument is a borrow of `text` that `push_str` reads while it writes to `text` (`code/compile_fail/ch05_two_phase_shared.rs`):

```rust
pub fn double(mut text: String) -> String {
    text.push_str(&text);
    text
}
```

```
error[E0502]: cannot borrow `text` as mutable because it is also borrowed as immutable
 --> compile_fail/ch05_two_phase_shared.rs:7:5
  |
7 |     text.push_str(&text);
  |     ^^^^^--------^-----^
  |     |    |        |
  |     |    |        immutable borrow occurs here
  |     |    immutable borrow later used by call
  |     mutable borrow occurs here
```

"immutable borrow later used by call" is the key label: the shared borrow is passed *into* the call, so it overlaps the activated mutable borrow. `push_str` may reallocate the buffer that `&text` points to while reading from it, so this is a real bug, not a limitation. The fix is to make the argument independent of `text` , as `to_uppercase()` did above (or `text.clone()` , or `text.repeat(2)` instead of the whole call).

5.3 Splitting borrows

Aliasing XOR mutation is a rule about *places*. Two mutable borrows may coexist when they point to places that cannot overlap. The question is always whether the compiler can *see* that they do not overlap.

Disjoint fields

Fields of a struct are distinct places, and the borrow checker tracks them separately. This compiles (`code/examples/ch05_split_borrows.rs`):

```rust
struct Document {
    title: String,
    words: Vec<String>,
}

fn tag_as_draft(doc: &mut Document) {
    let title = &mut doc.title; // borrows only `doc.title`
    let words = &mut doc.words; // borrows only `doc.words`
    title.push_str(" (draft)");
    words.push(String::from("draft"));
}
```

`&mut doc.title` borrows the place `(*doc).title`; `&mut doc.words` borrows `(*doc).words`. Neither is a prefix of the other, so there is no conflict.

Methods hide the split

Wrap the same two borrows in accessor methods and the program is rejected (`code/compile_fail/ch05_split_methods.rs`):

```
impl Document {
    pub fn title_mut(&mut self) -> &mut String {
        &mut self.title
    }

    pub fn words_mut(&mut self) -> &mut Vec<String> {
        &mut self.words
    }
}

pub fn tag_as_draft(doc: &mut Document) {
    let title = doc.title_mut();
    let words = doc.words_mut();
    title.push_str(" (draft)");
    words.push(String::from("draft"));
}
```

```
error[E0499]: cannot borrow `*doc` as mutable more than once at a time
  --> compile_fail/ch05_split_methods.rs:23:17
   |
22 |     let title = doc.title_mut();
   |                 --- first mutable borrow occurs here
23 |     let words = doc.words_mut();
   |                 ^^^ second mutable borrow occurs here
24 |     title.push_str(" (draft)");
   |     ----- first borrow later used here
```

The first mutable borrow is of `*doc`, the whole document. This is the signature-is-the-contract principle at work. The signature `fn title_mut(&mut self) -> &mut String` says: "give me exclusive access to all of `*self`, and I return a reference that may keep that access for as long as you use it". The caller is checked against the signature only, so it cannot know that the body touches only `title`. There is no syntax in stable Rust for "this method borrows only some fields of `self`".

There are three good ways out:

- **Borrow the fields directly** where they are visible (in the same module, or if they are public), as `tag_as_draft` did above.
- **One method, several results.** Inside the body the fields are visible, so a single method can split the borrow and return all the pieces at once. The caller still sees one borrow of `*doc`, and the two results share it:

```
impl Document {
    /// One method, one borrow of `self`, two disjoint results.
    fn parts_mut(&mut self) -> (&mut String, &mut Vec<String>) {
        (&mut self.title, &mut self.words)
    }
}

fn tag_as_final(doc: &mut Document) {
    let (title, words) = doc.parts_mut();
    title.push_str(" (final)");
```

```
    words.push(String::from("final"));
}
```

- **Destructure** the value, which binds one `&mut` reference per field:

```
fn tag_as_checked(doc: &mut Document) {
    let Document { title, words } = doc; // two `&mut` bindings, one per field
    title.push_str(" (checked)");
    words.push(String::from("checked"));
}
```

Closures get the same precision for free since edition 2021: a closure captures the individual places it uses, not the whole variable, so two closures that touch different fields can both be alive:

```
fn tag_with_closures(doc: &mut Document) {
    // Each closure captures only the field it uses (edition 2021 and later).
    let mut retitle = |suffix: &str| doc.title.push_str(suffix);
    let mut add_word = |word: &str| doc.words.push(word.to_owned());
    retitle(" (reviewed)");
    add_word("reviewed");
}
```

Slices and indices

Indexing is where the compiler stops being able to see disjointness (`code/compile_fail/ch05_split_index.rs`):

```
pub fn swap_first_two(words: &mut [String]) {
    let a = &mut words[0];
    let b = &mut words[1];
    std::mem::swap(a, b);
}
```

```
error[E0499]: cannot borrow `words[_]` as mutable more than once at a time
 --> compile_fail/ch05_split_index.rs:8:13
  |
7 |     let a = &mut words[0];
  |             -------------- first mutable borrow occurs here
8 |     let b = &mut words[1];
  |             ^^^^^^^^^^^^^^ second mutable borrow occurs here
9 |     std::mem::swap(a, b);
  |                    - first borrow later used here
  |
  = help: use `.split_at_mut(position)` to obtain two mutable non-overlapping sub-slices
```

The error names the place `words[_]`. The `_` is the borrow checker being honest: it does not track index values, which are run-time data, so every element of a slice is the same place to it. `words[0]` and `words[1]` might as well be `words[i]` and `words[j]`, which can be equal.

The standard library offers safe APIs that prove disjointness, either by construction or by a check at run time:

```
fn swap_ends(words: &mut [String]) {
    let last = words.len() - 1;
    let (head, tail) = words.split_at_mut(1); // [0, 1) and [1, len)
    std::mem::swap(&mut head[0], &mut tail[last - 1]);
}

fn swap_first_two(words: &mut [String]) {
    // Checks at run time that the indices are distinct and in bounds.
```

```
    if let Ok([a, b]) = words.get_disjoint_mut([0, 1]) {
        std::mem::swap(a, b);
    }
}
```

`split_at_mut(1)` returns two slices, `[0, 1)` and `[1, len)`, that cannot overlap by construction. (It is implemented with `unsafe` code, which is the subject of Chapter 13; its signature promises two independent borrows, and the promise is what you rely on.) `get_disjoint_mut`, stable since Rust 1.86, takes an array of indices, checks at run time that they are in bounds and pairwise distinct, and returns an array of `&mut` references or an error. Iterators such as `iter_mut()` and `chunks_mut()` give out disjoint `&mut` references one at a time. The output of the example:

```
Notes (draft) (final) (reviewed) (checked): ["draft", "final", "reviewed", "checked"]
["checked", "final", "reviewed", "draft"]
["final", "checked", "reviewed", "draft"]
```

5.4 Moves while borrowed, and borrows that cannot leave

Moving a borrowed value (E0505)

A move is a conflicting access, like a write: after the move, the old owner no longer owns the data, and the new owner may modify or free it. So you cannot move a value while a borrow of it is still going to be used (`code/compile_fail/ch05_move_while_borrowed.rs`):

```
pub fn parse_setting() -> Vec<String> {
    let line = String::from("timeout=30");
    let key = line.split('=').next().unwrap_or("");
    let mut lines = Vec::new();
    lines.push(line); // moves `line` into the vector
    println!("key: {key}");
    lines
}
```

```
error[E0505]: cannot move out of `line` because it is borrowed
  --> compile_fail/ch05_move_while_borrowed.rs:10:16
   |
 7 |     let line = String::from("timeout=30");
   |         ---- binding `line` declared here
 8 |     let key = line.split('=').next().unwrap_or("");
   |               ---- borrow of `line` occurs here
 9 |     let mut lines = Vec::new();
10 |     lines.push(line); // moves `line` into the vector
   |                ^^^^ move out of `line` occurs here
11 |     println!("key: {key}");
   |                     --- borrow later used here
```

The four labels form a timeline: where `line` was declared, where it was borrowed (`line.split('=')` returns slices of `line`, so `key` borrows it), where it was moved (into the vector), and where the borrow is used later (the `println!`). The error exists only because of the last label: delete the `println!` and the program compiles, because `key` is dead before the move.

The fixes follow from the timeline. Use the borrow before the move, or move after the last use. Or stop borrowing: give `key` its own copy, which is what `code/examples/ch05_ownership_fixes.rs` does:

```rust
fn parse_setting() -> (String, Vec<String>) {
    let line = String::from("timeout=30");
    let key = line.split('=').next().unwrap_or("").to_owned(); // an owned copy
    let mut lines = Vec::new();
    lines.push(line); // fine: `key` does not borrow `line`
    (key, lines)
}
```

A third option is to borrow from the new owner (`lines[0].split('=')`) after the move.

Returning a reference to a local (E0515)

A function cannot return a borrow of data it owns, because that data is dropped when the function returns. The compiler has two wordings for this error, depending on whether you return the reference directly or a value that contains one; `code/compile_fail/ch05_return_local.rs` shows both (two functions, one error each):

```rust
pub fn first_word(path: &str) -> &str {
    let text = format!("contents of {path}");
    text.split_whitespace().next().unwrap_or("")
}

pub fn whole_text(path: &str) -> &String {
    let text = format!("contents of {path}");
    &text
}
```

```
error[E0515]: cannot return value referencing local variable `text`
 --> compile_fail/ch05_return_local.rs:8:5
  |
8 |     text.split_whitespace().next().unwrap_or("")
  |     ----^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |     |
  |     returns a value referencing data owned by the current function
  |     `text` is borrowed here

error[E0515]: cannot return reference to local variable `text`
  --> compile_fail/ch05_return_local.rs:13:5
   |
13 |     &text
   |     ^^^^^ returns a reference to data owned by the current function
```

"returns a value referencing data owned by the current function" is the essence of E0515. Notice what the signature says: by elision rule 2 (Chapter 3), the result of `first_word(path: &str) -> &str` borrows from `path`. The body tries to return a borrow of `text` instead. No lifetime annotation can fix that, because annotations describe what a reference may point to; they never make anything live longer (Chapter 1). The fix is to change what is returned: an owned value,

```rust
fn first_word(path: &str) -> String {
    let text = format!("contents of {path}");
    text.split_whitespace().next().unwrap_or("").to_owned()
}
```

or a borrow of something the caller owns (take the text as a parameter), or a `Cow<'_, str>` that is sometimes borrowed and sometimes owned (Chapter 14).

A borrow of a temporary (E0716)

The third error of this family has a twist: the value that dies is not a variable at all. From `code/compile_fail/ch05_temporary_dropped.rs`:

```
pub fn config_name() -> usize {
    let name: &str = String::from("config.toml").as_str();
    name.len()
}
```

```
error[E0716]: temporary value dropped while borrowed
 --> compile_fail/ch05_temporary_dropped.rs:8:22
  |
8 |     let name: &str = String::from("config.toml").as_str();
  |                      ^^^
```

```
    }

    fn find(&self, c: char) -> Option<usize> {
        self.0.find(c)
    }
}

impl Drop for Noisy {
    fn drop(&mut self) {
        println!("  drop {}", self.0);
    }
}
```

The first experiment is the default rule:

```
println!("1. end of statement");
let len = make("a").len();
println!("  after the let, len = {len}");
```

```
1. end of statement
  make a
  drop a
  after the let, len = 1
```

The `Noisy` is created, `len` reads it, and it is dropped at the `;` of the `let`, before the next statement. `len` is a `usize`, so nothing refers to the dropped value, and all is well. In `ch05_temporary_dropped.rs`, the result of `as_str()` was a reference into the temporary: same timing, but now a dangling reference, and the compiler rejects it.

Temporary lifetime extension

There is one exception that you use all the time without noticing. When the initializer of a `let` borrows a temporary directly, the temporary lives until the end of the enclosing block, exactly as if it had a name:

```
println!("2. lifetime extension");
{
    let held = &make("b");
    println!("  after the let, still holding {}", held.0);
}
println!("  after the block");
```

```
2. lifetime extension
  make b
  after the let, still holding b
  drop b
  after the block
```

This is **temporary lifetime extension**. It is a *syntactic* rule: it applies when the borrow of the temporary appears in an "extending" position of the initializer. `code/examples/ch05_lifetime_extension.rs` collects the positions that rustc 1.99 extends:

```
let direct = &load("direct"); // operand of `&`
let entry = Entry { key: &load("lit") }; // field of a struct literal
let pair = (&load("tuple"), 1); // element of a tuple
let list = [&load("array")]; // element of an array
let block = { &load("block") }; // tail of a block
let branch = if quiet { &load("then") } else { &load("else") }; // branches
```

```
let arm = match quiet {
    true => &load("yes"),
    false => &load("no"), // arms of a `match`
};
let wrapped = Some(&load("variant")); // tuple-variant constructor
```

```
<direct> <lit> <tuple> <array>
<block> <else> <no> Some("<variant>")
```

Extension goes through the operand of `&`, the fields of struct, tuple and array expressions, the tail of a block, the branches of `if` and `match`, and (in rustc 1.99) the arguments of a tuple-struct or enum-variant constructor such as `Some`. It does not go through a method call or a function call, because the compiler cannot know what the function does with the reference. That is why `String::from("config.toml").as_str()` was an error, and why this second function in `ch05_temporary_dropped.rs` is one too:

```
pub fn config_path() -> usize {
    let path = std::convert::identity(&String::from("/etc/app"));
    path.len()
}
```

```
error[E0716]: temporary value dropped while borrowed
  --> compile_fail/ch05_temporary_dropped.rs:15:40
   |
15 |     let path = std::convert::identity(&String::from("/etc/app"));
   |                                        ^^^^^^^^^^^^^^^^^^^^^^^^ - temporary value is
freed at the end of this statement
   |                                        |
   |                                        creates a temporary value which is freed while
still in use
16 |     path.len()
   |     ---- borrow later used here
   |
help: consider using a `let` binding to create a longer lived value
   |
15 ~     let binding = String::from("/etc/app");
16 ~     let path = std::convert::identity(&binding);
   |
```

The fix for both is the help's suggestion: a named variable, which `ch05_ownership_fixes.rs` shows next to an extended temporary:

```
fn config_name() -> usize {
    let owned = String::from("config.toml"); // a named value lives to the block end
    let name: &str = owned.as_str();
    let extended: &String = &String::from("config.toml"); // lifetime-extended
    name.len() + extended.len()
}
```

Temporaries in `match` and `if let`

A temporary created in the *scrutinee* of a `match` (the expression being matched) lives until the end of the whole `match`, in every edition:

```
println!("3. match scrutinee");
match make("c").len() {
    n => println!("  in the arm, n = {n}"),
```

```
}
println!("  after the match");
```

```
3. match scrutinee
  make c
  in the arm, n = 1
  drop c
  after the match
```

The arm runs while `c` is still alive, although the value being matched, a `usize`, has nothing to do with it. The reason is that patterns can bind references into the scrutinee, so the compiler keeps its temporaries alive for all the arms; it does not check whether a particular `match` needs that.

For values without a destructor this timing is invisible. It becomes visible when the temporary holds a borrow *and* has a `Drop` impl, because dropping it then counts as a use of the borrow (more on this in the next section). Here is such a type, a view of a list of lines (`code/compile_fail/ch05_if_let_2021.rs`):

```
impl<'a> Lines<'a> {
    pub fn new(lines: &'a Vec<String>) -> Self {
        Lines { lines }
    }

    pub fn first(&self) -> Option<&'a str> {
        self.lines.first().map(|line| line.as_str())
    }
}

impl Drop for Lines<'_> {
    fn drop(&mut self) {}
}
```

and a function that matches on its first line, and adds a line when there is none. The compiler rejects it, in every edition (`code/compile_fail/ch05_match_scrutinee.rs`):

```
pub fn show_first(lines: &mut Vec<String>) {
    match Lines::new(lines).first() {
        Some(first) => println!("first line: {first}"),
        None => lines.push(String::from("(empty)")),
    }
}
```

```
error[E0502]: cannot borrow `*lines` as mutable because it is also borrowed as immutable
  --> compile_fail/ch05_match_scrutinee.rs:27:17
   |
25 |     match Lines::new(lines).first() {
   |           ------------------
   |           |          |
   |           |          immutable borrow occurs here
   |           a temporary with access to the immutable borrow is created here ...
26 |         Some(first) => println!("first line: {first}"),
27 |         None => lines.push(String::from("(empty)")),
   |                 ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
28 |     }
   |     - ... and the immutable borrow might be used here, when that temporary is dropped
and runs the `Drop` code for type `Lines`
```

The diagnostic now has a two-part story, linked by "…". First: "a temporary with access to the immutable borrow is created here ...", pointing at `Lines::new(lines)` . Then, at the closing brace of the `match` : "… and the immutable borrow might be used here, when that temporary is dropped

and runs the `Drop` code for type `Lines`". In between, the `None` arm wants to push to the vector. The `Option<&str>` returned by `first()` is not the problem (in the `None` arm there is no `&str`); the temporary `Lines` is. The fix is to end the temporary's life before the `match`, with a `let` (`code/examples/ch05_temporaries_2024.rs`):

```
fn show_first_match(lines: &mut Vec<String>) {
    let first = Lines::new(lines).first(); // the temporary dies at this `;`
    match first {
        Some(first) => println!("first line: {first}"),
        None => lines.push(String::from("(empty)")),
    }
}
```

`if let` used to follow the same rule as `match`, and edition 2024 changed it. Here is the experiment with `Noisy`, once with a match that succeeds and once with one that fails:

```
println!("4. if let scrutinee");
if let Some(i) = make("dog").find('o') {
    println!("  in the then-branch, i = {i}");
}
if let Some(i) = make("cat").find('o') {
    println!("  in the then-branch, i = {i}");
} else {
    println!("  in the else-branch");
}
```

```
4. if let scrutinee
  make dog
  in the then-branch, i = 1
  drop dog
  make cat
  drop cat
  in the else-branch
```

The scrutinee's temporaries live through the then-branch (`dog` is dropped after it) but are dropped *before* the else-branch runs (`cat` is dropped first). In edition 2021 they lived until the end of the whole `if let`, `else` included. The `Lines` version of `show_first` written with `if let` shows the difference. Compiled with edition 2021, it is rejected with the same story as the `match`:

```
pub fn show_first(lines: &mut Vec<String>) {
    if let Some(first) = Lines::new(lines).first() {
        println!("first line: {first}");
    } else {
        lines.push(String::from("(empty)"));
    }
}
```

```
error[E0502]: cannot borrow `*lines` as mutable because it is also borrowed as immutable
  --> compile_fail/ch05_if_let_2021.rs:31:9
   |
28 |     if let Some(first) = Lines::new(lines).first() {
   |                          -----------------
   |                          |          |
   |                          |          immutable borrow occurs here
   |                          a temporary with access to the immutable borrow is created
here ...
...
31 |         lines.push(String::from("(empty)"));
   |         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
32 |     }
```

```
33 | }
   | - ... and the immutable borrow might be used here, when that temporary is dropped and
runs the `Drop` code for type `Lines`
```

The same function, unchanged, compiles in edition 2024; it is part of `ch05_temporaries_2024.rs`.

> **NOTE**
>
> Edition 2024 (Rust 1.85 and later) drops the temporaries of an `if let` scrutinee before the `else` branch. The edition guide describes the change at https://doc.rust-lang.org/edition-guide/rust-2024/temporary-if-let-scope.html. The lint `if_let_rescope` (in the `rust_2024_compatibility` group) flags the `if let` expressions whose behavior changes when you migrate a crate. A `match` keeps the old scope in every edition, and so does `while let` (Chapter 12 shows what that means for lock guards).

Temporaries in tail expressions

The second change of edition 2024 concerns the *tail expression* of a block, the final expression without a `;` that gives the block its value. In edition 2021, temporaries created there were dropped *after* the block's local variables, so a temporary that borrowed a local outlived it. Edition 2024 drops them *before* the locals:

```
fn tail_expression() -> usize {
    let _local = make("local");
    make("tail").len() // edition 2024: `tail` is dropped before `_local`
}
```

```
5. tail expression
  make local
  make tail
  drop tail
  drop local
  returned 4
```

`tail` is dropped before `_local`, although `_local` was created first. Edition 2021 printed the two drops in the opposite order. When the temporary borrows the local and has a destructor, the old order is a compile error. Here is a reading view with a `Drop` impl (`code/compile_fail/ch05_tail_expr_2021.rs`):

```
/// A view of a text. It has a destructor, so dropping it counts as a use of
/// the borrow it holds.
pub struct Reader<'a> {
    text: &'a str,
}

impl<'a> Reader<'a> {
    pub fn new(text: &'a str) -> Self {
        Reader { text }
    }

    pub fn word_count(&self) -> usize {
        self.text.split_whitespace().count()
    }
}

impl Drop for Reader<'_> {
    fn drop(&mut self) {}
}
```

and a function whose tail expression creates one. Compiled with edition 2021, it is rejected:

```
pub fn count_words() -> usize {
    let text = String::from("one two three");
    Reader::new(&text).word_count()
}
```

```
error[E0597]: `text` does not live long enough
  --> compile_fail/ch05_tail_expr_2021.rs:32:17
   |
31 |     let text = String::from("one two three");
   |         ---- binding `text` declared here
32 |     Reader::new(&text).word_count()
   |     ------------^^^^^-
   |     |           |
   |     |           borrowed value does not live long enough
   |     a temporary with access to the borrow is created here ...
33 | }
   | -
   | |
   | `text` dropped here while still borrowed
   | ... and the borrow might be used here, when that temporary is dropped and runs the
`Drop` code for type `Reader`
   |
   = note: the temporary is part of an expression at the end of a block;
           consider forcing this temporary to be dropped sooner, before the block's local
variables are dropped
help: for example, you could save the expression's value in a new local variable `x` and
then make `x` be the expression at the end of the block
   |
32 |     let x = Reader::new(&text).word_count(); x
   |     +++++++                                +++
```

The labels at the closing brace show the problem in edition 2021's order: first "`text` dropped here while still borrowed", then "… and the borrow might be used here, when that temporary is dropped". The note and the help describe the workaround that edition 2021 code is full of: store the result in a local and make that local the tail expression. In edition 2024 the function compiles as written (it is in `ch05_temporaries_2024.rs`), because the `Reader` is dropped first. The change is described at https://doc.rust-lang.org/edition-guide/rust-2024/temporary-tail-expr-scope.html, and the migration lint is `tail_expr_drop_order`.

Here are the rules of this section in one table:

A temporary created in…	is dropped (edition 2024)	in edition 2021
a statement, or a `let` without extension	at the end of the statement	same
a `let` initializer, extended	at the end of the block	same
a `match` scrutinee	at the end of the `match`	same
a `while let` scrutinee	at the end of each iteration	same
an `if let` scrutinee	before the `else` branch	at the end of the `if let`
the tail expression of a block	before the block's locals	after the block's locals

5.6 Drop order and the drop check

The order of drops

Rust drops values at precise, predictable points. Local variables are dropped at the end of their scope, in the *reverse* order of their declaration. The fields of a struct are dropped in the order they are *declared*, after the struct's own `Drop` impl (if any) has run. A `let _ = expr;` does not bind anything: `_` is a pattern that matches without moving, so the value of `expr` stays a temporary and is dropped at the end of the statement. `code/examples/ch05_drop_order.rs` shows all three:

```
struct Pair {
    first: Noisy,
    second: Noisy,
}
```

```
println!("locals and fields:");
{
    let _a = Noisy("a");
    let _b = Noisy("b");
    let pair = Pair {
        first: Noisy("pair.first"),
        second: Noisy("pair.second"),
    };
    let _ = Noisy("ignored"); // `_` is not a binding: dropped right away
    println!("  end of block ({} and {})", pair.first.0, pair.second.0);
}
```

```
locals and fields:
  drop ignored
  end of block (pair.first and pair.second)
  drop pair.first
  drop pair.second
  drop b
  drop a
```

Dropping is a use

A destructor is code that runs at the end of a scope, and it can read the value's fields, including the references inside them. So when a value of a type with a `Drop` impl is dropped, the borrow checker treats the drop as a *use* of every borrow the value holds. This is the **drop check**. Here is a report that prints a summary when it goes away:

```
pub struct Report<'a> {
    lines: Vec<&'a str>,
}

impl Drop for Report<'_> {
    fn drop(&mut self) {
        println!("report: {} lines", self.lines.len());
    }
}
```

Declare the report first and the text second, and the program is rejected (`code/compile_fail/ch05_drop_check.rs`):

```
pub fn summarize() {
    let mut report = Report { lines: Vec::new() };
    let text = String::from("first line\nsecond line");
```

```
    report.lines.extend(text.lines());
}
```

```
error[E0597]: `text` does not live long enough
  --> compile_fail/ch05_drop_check.rs:21:25
   |
20 |     let text = String::from("first line\nsecond line");
   |         ---- binding `text` declared here
21 |     report.lines.extend(text.lines());
   |                         ^^^^ borrowed value does not live long enough
22 | }
   | -
   | |
   | `text` dropped here while still borrowed
   | borrow might be used here, when `report` is dropped and runs the `Drop` code for type
`Report`
   |
   = note: values in a scope are dropped in the opposite order they are defined
```

All the action is at the closing brace. Variables are dropped in reverse order, so `text` is dropped first ("`text` dropped here while still borrowed"); then `report` is dropped, and "borrow might be used here, when `report` is dropped and runs the `Drop` code for type `Report`". The note states the rule that decides it: "values in a scope are dropped in the opposite order they are defined". If the compiler accepted this program, `Report::drop` could read the `&str`s in `self.lines`, which point into a freed `String`. This destructor happens to look only at the length, but the borrow checker does not read destructors: it goes by the existence of the `impl Drop`, as it goes by signatures everywhere else.

The fix is the order of declaration. Declare the data before the value that borrows it, so that the borrower is dropped first:

```
println!("a report declared after its text:");
{
    let text = String::from("first line\nsecond line");
    let mut report = Report { lines: Vec::new() };
    report.lines.extend(text.lines());
} // `report` is dropped first, while `text` is still alive
```

```
a report declared after its text:
  report: 2 lines, first "first line"
```

Why `Vec<&str>` is more relaxed

The same code with a plain `Vec<&str>` instead of a `Report` compiles in the "wrong" order:

```
println!("a plain Vec<&str> declared before its text:");
{
    let mut lines: Vec<&str> = Vec::new();
    let text = String::from("first line\nsecond line");
    lines.extend(text.lines());
    println!("  {} lines", lines.len());
} // `text` is dropped first; `Vec`'s destructor never reads the `&str`s
```

```
a plain Vec<&str> declared before its text:
  2 lines
```

`Vec` has a destructor too (it frees its buffer), so why is its drop not a use of the `&str`s? Because the standard library tells the compiler that `Vec`'s destructor does not read the borrowed data of its

elements. It only drops the elements, and dropping a `&str` does nothing. The promise is made with an unstable attribute, `#[may_dangle]`, that only the standard library can use on stable Rust; Chapter 13 explains it at the level a user of `Vec` needs. `Box` makes the same promise, and `Option` has no destructor of its own.

A struct of your own is just as relaxed as long as it has no `Drop` impl, because then its drop glue only drops its fields one by one:

```
/// The same fields, but no `Drop` impl.
struct Notes<'a> {
    lines: Vec<&'a str>,
}
```

```
println!("a struct without Drop declared before its text:");
{
    let mut notes = Notes { lines: Vec::new() };
    let text = String::from("first line\nsecond line");
    notes.lines.extend(text.lines());
    println!("  {} lines", notes.lines.len());
} // no `impl Drop for Notes`: dropping `notes` reads nothing
```

```
a struct without Drop declared before its text:
  2 lines
```

So adding an `impl Drop` to a type that holds references is a change to its public contract: it makes every user's declaration order matter. If the destructor does not need the borrowed data, consider whether the type needs a destructor at all, or whether the destructor can live in a field that holds only owned data.

5.7 The limits of NLL

The borrow checker that you use is called NLL, for non-lexical lifetimes (Chapter 1). It computes, for each borrow, a region: the set of program points where the borrow may still be used. It is precise enough that almost every correct program you write is accepted. Almost.

NLL problem case #3

Here is a function that looks up a word in a cache, and computes and inserts the value when it is missing (`code/compile_fail/ch05_nll_case3.rs`):

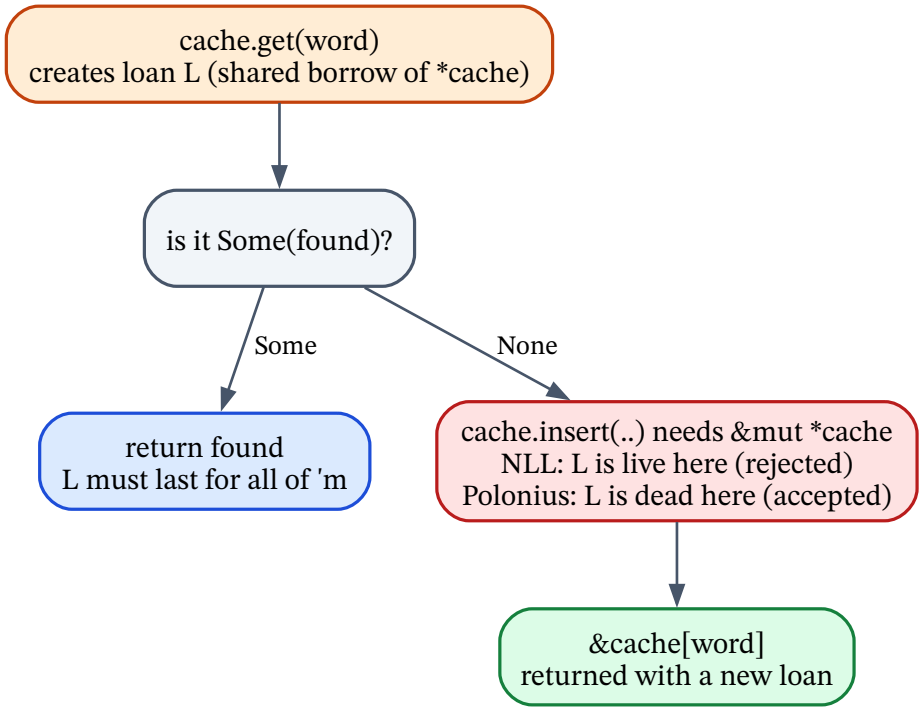
```
pub fn cached<'m>(cache: &'m mut HashMap<String, String>, word: &str) -> &'m String {
    if let Some(found) = cache.get(word) {
        return found;
    }
    cache.insert(word.to_owned(), word.to_uppercase());
    &cache[word]
}
```

The function is correct. If the lookup succeeds, it returns the borrowed value and never touches the map again. If it fails, there is no `found`, so nothing refers into the map when `insert` modifies it. Stable Rust rejects it anyway:

```
error[E0502]: cannot borrow `*cache` as mutable because it is also borrowed as immutable
  --> compile_fail/ch05_nll_case3.rs:13:5
   |
 9 | pub fn cached<'m>(cache: &'m mut HashMap<String, String>, word: &str) -> &'m String {
   |               -- lifetime `'m` defined here
10 |     if let Some(found) = cache.get(word) {
   |                          ----- immutable borrow occurs here
11 |         return found;
   |                ----- returning this value requires that `*cache` is borrowed for `'m`
12 |     }
13 |     cache.insert(word.to_owned(), word.to_uppercase());
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
```

Read the labels in order. "lifetime `'m` defined here": the caller picks `'m`, and it covers the whole call, from the caller's point of view. "immutable borrow occurs here": `cache.get(word)` borrows the map. "returning this value requires that `*cache` is borrowed for `'m`": because `found` is returned, the borrow must last for all of `'m`. And then "mutable borrow occurs here" at the `insert`.

The problem is in the third label. NLL represents the borrow by one region, and the constraint "this region outlives `'m`" does not depend on the path taken: a region that must outlive a lifetime parameter of the function contains every point of the function body. So the borrow is considered live at the `insert` too, although `insert` can only be reached on the path where nothing was returned:



NLL problem case #3. The loan created by `cache.get` must last for all of `'m` because it is returned on the `Some` path; NLL therefore considers it live on the `None` path too, at the `insert`. Polonius sees that the loan cannot reach the `insert`.

This example has a name. The RFC that introduced NLL listed it as "Problem case #3: conditional control flow across functions", one of the cases that NLL would *not* solve: https://rust-lang.github.io/rfcs/2094-nll.html.

Polonius

Polonius is a reformulation of the borrow checker designed to accept such programs. Roughly, instead of asking at each point "is this region live here?", it asks "which loans can reach this point, along the paths that actually lead here?". At the `insert`, the only path that arrives there is the `None` path, on which the loan from `cache.get` is not held by any live reference, so there is no conflict.

> **NOTE**
>
> As of this writing Polonius is not part of stable Rust. Its first version, called Polonius Alpha, is enabled by default on nightly compilers since August 2026 (announced at https://blog.rust-lang.org/2026/08/04/enabling-polonius-alpha-on-nightly/, which says the team is "aiming to stabilize prior to the end of the year"). The nightly of 2026-10-08 (`rustc 1.101.0-nightly`) compiles `ch05_nll_case3.rs` without errors, and the flag `-Zpolonius=off` brings the E0502 back. Check the current status before you rely on it.

You can try it yourself, from the `code/` directory:

```
rustup toolchain install nightly
rustc +nightly --edition 2024 --crate-type lib --emit=metadata \
    --out-dir target compile_fail/ch05_nll_case3.rs
```

Workarounds on stable Rust

Until then, you rewrite the function so that no borrow is held on the path that mutates. The entry API does the lookup and the insertion with one mutable borrow (`code/examples/ch05_nll_workarounds.rs`):

```
fn cached_entry<'m>(cache: &'m mut HashMap<String, String>, word: &str) -> &'m String {
    cache
        .entry(word.to_owned()) // allocates the key even on a hit
        .or_insert_with(|| word.to_uppercase())
}
```

It costs a `String` allocation for the key even when the word is already there. The other classic rewrite asks a question whose answer is a `bool`, so that no borrow survives the test, and looks up the value a second time:

```
fn cached_twice<'m>(cache: &'m mut HashMap<String, String>, word: &str) -> &'m String {
    if !cache.contains_key(word) {
        // `contains_key` returned a `bool`: no borrow survives the test.
        cache.insert(word.to_owned(), word.to_uppercase());
    }
    &cache[word] // a second lookup
}
```

```
ALPHA
ALPHA
BETA
BETA
2 entries
```

The recognizable shape of the problem is a function that receives a `&mut`, returns a borrow obtained from it on one path, and uses it mutably on another path. If an error carries the label "returning this value requires that ... is borrowed for `'a`", and the conflicting access is on a path where that value was

never returned, you are looking at case #3. The usual tools are the ones above: an API that does both steps at once (`entry`), a test that returns no borrow (`contains_key`, a `bool`, an index), or returning an index or an owned value instead of a reference.

5.8 Common confusions

"**A `&mut` parameter can be used only once.**" It can be used as often as you like, as long as each use is a reborrow: calls to functions that take `&mut T`, method calls, and `let` with a written type all reborrow. Only a plain move (`let r2 = r;`, a generic parameter, a wrapper such as `Option<&mut T>`) gives the reference away.

"**Two-phase borrows change the order of evaluation.**" They do not. The receiver is still evaluated first; only the *activation* of its mutable borrow waits for the call. That is why the arguments may read the receiver but not keep a borrow of it into the call.

"**Lifetime extension keeps temporaries alive when I need it.**" Only in the syntactic positions listed above. A temporary passed through a function or a method call is never extended, whatever the function returns.

"**I'll call `drop` explicitly to fix the drop order.**" For the drop check, the order of *declaration* matters, and an explicit `drop` does not change it. This program (`code/compile_fail/ch05_drop_check_explicit.rs`) is still rejected:

```rust
pub fn summarize() {
    let mut report = Report { lines: Vec::new() };
    let text = String::from("first line\nsecond line");
    report.lines.extend(text.lines()); // may panic (allocation)
    drop(report);
}
```

```
error[E0597]: `text` does not live long enough
  --> compile_fail/ch05_drop_check_explicit.rs:19:25
   |
18 |     let text = String::from("first line\nsecond line");
   |         ---- binding `text` declared here
19 |     report.lines.extend(text.lines()); // may panic (allocation)
   |                         ^^^^ borrowed value does not live long enough
20 |     drop(report);
21 | }
   | -
   | |
   | `text` dropped here while still borrowed
   | borrow might be used here, when `report` is dropped and runs the `Drop` code for type
`Report`
   |
   = note: values in a scope are dropped in the opposite order they are defined
```

And rightly so: if `extend` panicked, unwinding would drop the variables in reverse order of declaration, `text` and then `report`, whose destructor could read the freed lines. Declare `text` first.

"**The compiler rejected it, so it must be unsafe.**" Usually, yes. But the borrow checker is conservative by design, and case #3 is a correct program that stable Rust rejects. When you are sure a program is correct, look for the shape of case #3 before you look for a bug.

5.9 Summary

- A `&mut T` is not `Copy`. Where the expected type is known to be `&mut` (function arguments, `let` with a type, method receivers), the compiler *reborrows* it (`&mut *r`) instead of moving it; with generics and wrappers such as `Option<&mut T>` it moves, and you reborrow by hand (`&mut *r`, `as_deref_mut()`).
- While a reborrow is live, the original reference is frozen. Borrows are tracked per place (`*words`, `doc.title`, `words[_]`).
- Two-phase borrows reserve the `&mut` of a method receiver, an implicit argument reborrow, or an overloaded compound assignment, and activate it when the call starts: reads in between are allowed, a shared borrow passed into the call is not. A `&mut` you write yourself is never two-phase.
- Disjoint fields can be borrowed mutably at the same time; accessor methods cannot express that, because the signature borrows all of `self`. Return several borrows from one method, or destructure. For slices, use `split_at_mut`, `get_disjoint_mut` (1.86) or `iter_mut`.
- E0505: a value is moved while a borrow of it is still used. E0515: a function returns a borrow of its own local. E0716: a borrow outlives the temporary it points to.
- Temporaries die at the end of their statement unless lifetime extension applies (a borrow in an extending position of a `let`). `match` and `while let` scrutinees keep their temporaries to the end; in edition 2024, `if let` drops them before `else`, and tail expressions drop them before the block's locals.
- Locals are dropped in reverse order of declaration, fields in declaration order. The drop check treats dropping a value whose type has a `Drop` impl as a use of its borrows; `Vec<&T>` and types without a `Drop` impl are exempt.
- NLL rejects some correct programs, most famously a borrow returned on one path and a mutation on another (problem case #3). Polonius, on by default on nightly since August 2026, accepts it; on stable, use the entry API or a test that holds no borrow.

5.10 Exercises

1. Without compiling, predict which of these calls compile when `buf` has type `&mut String`: `fill(buf); fill(buf);` with `fn fill(b: &mut String)`; `show(buf); show(buf);` with `fn show<T: std::fmt::Debug>(x: T)`; `let b2 = buf; fill(b2); fill(buf);`. Then check your answers and fix the ones that fail with the smallest change. *Hint:* ask, at each call, whether the compiler knows that the expected type is a `&mut` reference.
2. Give `Document` from `ch05_split_borrows.rs` a third field, `errors: Vec<String>`, and write a method that returns mutable references to `words` and `errors` at the same time. Then write a function that moves every word that contains a digit from `words` to `errors`. *Hint:* the method's body can borrow the two fields separately; `retain` and `push` on the two results do the rest.
3. Copy `ch05_temporaries.rs` to a scratch directory and compile it with `rustc --edition 2021`. Which lines of the output change, and why? Then delete the `impl Drop for Lines` from `ch05_match_scrutinee.rs` and compile it again: what happens, and what does that tell you about

when temporary scopes matter? *Hint:* without a destructor, dropping the temporary is not a use of the borrow it holds.

4. Make `ch05_drop_check.rs` compile in three different ways: by changing the order of declarations, by changing the field type so that `Report` owns its lines, and by moving the printing out of the `Drop` impl. Which one would you choose for a type in a library, and why? *Hint:* think about what each change costs the *users* of `Report`.
5. Write `fn longest_or_insert<'v>(words: &'v mut Vec<String>) -> &'v String`, which returns the longest word if it has more than five letters and otherwise pushes `"(none)"` and returns a reference to it. Write it first in the natural way, observe the error, then write a version that compiles on stable Rust. *Hint:* find the *index* of the longest word first; an index holds no borrow.